UNIVERZITET U BEOGRADU
MATEMATIČKI FAKULTET

Propratni rad iz predmeta

Dizajn i analiza algoritama

Simulacija rutiranja u mreži korišćenjem TRIE strukture i Dijkstra algoritma

Student:

Marko Rašković

Beograd, 2026.

Sadržaj

Sažetak **1**

1 Formulacija problema i motivacija **1**

2 Model mreže i IP rutiranja **1**

3 Trie struktura za pronalaženje IP prefiksa **1**

4 Dijkstra algoritam za najkraću putanju **2**
4.1 Povezivanje trie strukture i Dijkstra algoritma 2

5 Implementacija programa **2**
5.1 Primer ruting tabele 3

6 Analiza vremenske složenosti **3**

7 Primer izvršavanja programa **4**

8 Zaključak **4**

Sažetak

U ovom radu prikazana je jednostavna simulacija rutiranja paketa kroz mrežu rutera. Problem se sastoji iz dva povezana koraka: prvo se na osnovu odredišne IP adrese određuje kojoj mreži adresa pripada, a zatim se kroz graf rutera traži najkraća putanja do rutera koji vodi ka toj mreži. Za prvi korak koristi se trie struktura, jer prirodno predstavlja binarne prefikse IP adresa. Za drugi korak koristi se Dijkstra algoritam, jer računa najkraće puteve grafu sa nenegativnim težinama.

1 Formulacija problema i motivacija

U računarskoj mreži paket se šalje od početnog rutera ka odredišnoj IP adresi. Ruter ne mora znati tačnu putanju do svake pojedinačne IP adrese. Umesto toga, on koristi ruting tabelu koja sadrži mrežne prefikse, na primer 192.168.2.0/24. Ako odredišna adresa pripada nekom prefiksu, paket se prosleđuje prema ruteru koji je zadužen za taj deo mreže.

Formalno, neka je $G = (V, E)$ neusmeren tezinski graf, gde su čvorovi ruteri, a grane veze između rutera. Svaka grana ima cenu koja može predstavljati kašnjenje, opterećenje ili administrativnu cenu linka. Dodatno postoji skup prefiksa P, gde svakom prefiksu odgovara jedan izlazni ili odredišni ruter. Za dati početni ruter s i odredišnu IP adresu a potrebno je pronaći ruter t koji odgovara najdužem prefiksu adrese a, a zatim najjeftiniju putanju od s do t.

Motivacija za ovakav pristup dolazi iz realnog IP rutiranja. Pravilo najdužeg poklapanja prefiksa je standardni način izbora rute: ako jedna adresa odgovara i prefiksu 10.0.0.0/8 i prefiksu 10.1.2.0/24, bira se konkretniji prefiks /24. Zbog toga je potrebno imati strukturu koja brzo prolazi kroz bitove IP adrese i pamti poslednji pronađeni prefiks.

2 Model mreže i IP rutiranja

U programu je mreža predstavljena običnim rečnikom u Pythonu. Ključ rečnika je naziv rutera, a vrednost je lista suseda i cena veze do svakog suseda. Time se dobija lista susedstva, što je jednostavan i efikasan način čuvanja retkog grafa.

Element modela	**Značenje u programu**
Ruter	Čvor grafa, npr. R1, R2, R3
Veza	Grana između dva rutera
Cena veze	Težina grane koju Dijkstra minimizuje
IP prefiks	Mreža zapisana u obliku adresa/maska, npr. 192.168.2.0/24
Odredišni ruter	Ruter koji je povezan sa pronađenim prefiksom

3 Trie struktura za pronalaženje IP prefiksa

Trie je stablo u kome se svaka grana može posmatrati kao jedan karakter ili jedan bit. Kod IPv4 adresa svaka adresa ima 32 bita, pa se prefiks /24 čuva tako što se u stablo ubace prva 24 bita mrežne adrese. Čvor na kome se prefiks završava čuva informaciju o mreži i ruteru kome ta mreža pripada.

Pretraga najdužeg prefiksa radi tako što se od korena trie stabla prate bitovi odredišne IP adrese. Tokom prolaska pamti se poslednji čvor koji predstavlja validan prefiks. Kada dalji bit ne postoji u stablu ili kada se prođe svih 32 bita, vraća se poslednji zapamćeni prefiks. Tako se dobija pravilo najdužeg poklapanja.

```
najduzi_prefiks(ip):
    bitovi = pretvori_ip_u_32_bita(ip)
    cvor = koren
    poslednji = nema
    za svaki bit u bitovi:
        ako bit ne postoji medju decom cvora: prekini
        cvor = cvor.deca[bit]
        ako cvor sadrzi ruter: poslednji = cvor
    vrati poslednji
```

4 Dijkstra algoritam za najkraću putanju

Nakon što trie odredi ka kom ruteru treba slati paket, ostaje pitanje kojom putanjom stići do tog rutera. Pošto veze imaju nenegativne cene, prirodan izbor je Dijkstra algoritam. Algoritam uvek obrađuje trenutno najbliži neobrađeni ruter i pokušava da preko njega popravi rastojanja do njegovih suseda.

U implementaciji se koristi prioritetni red preko modula `heapq`. Element reda sadrži trenutnu cenu, naziv rutera i putanju do tog rutera. Kada se iz reda izvadi odredišni ruter, pronađena putanja je najkraća jer su sve težine nenegativne.

```
dijkstra(graf, pocetak, kraj):
    red = [(0, pocetak, [pocetak])]
    poseceni = prazan skup
    dok red nije prazan:
        cena, ruter, putanja = izvadi_najmanji(red)
        ako je ruter vec posecen: nastavi
        ako je ruter == kraj: vrati cenu i putanju
        za svakog suseda:
            ubaci novu cenu i novu putanju u red
```

4.1 Povezivanje trie strukture i Dijkstra algoritma

Cela simulacija se zato može posmatrati kao kompozicija dva algoritma. Trie daje odgovor na pitanje gde se nalazi odredišna mreža, a Dijkstra daje odgovor na pitanje kako najjeftinije doći do tog rutera. Ako trie ne pronađe prefiks, program ispisuje da ruta ne postoji. Ako prefiks postoji, ali graf nije povezan, program može prijaviti da ne postoji putanja kroz mrežu.

Korak	Algoritam	Rezultat
1	Trie pretraga	Najduži IP prefiks i odredišni ruter
2	Dijkstra	Najkraća putanja od početnog do odredišnog rutera
3	Ispis	Prefiks, putanja i ukupna cena

5 Implementacija programa

Program je napisan u Pythonu i podeljen je na nekoliko jednostavnih celina. Klasa `TrieCvor` predstavlja jedan čvor stabla. Svaki čvor ima rečnik dece, moguće ime rutera i zapamćen prefiks.

Klasa `IPTrie` sadrži metode za ubacivanje prefiksa i pretragu najdužeg poklapanja. Funkcija `dijkstra` radi nad grafom rutera.

Deo programa	Uloga
`TrieCvor`	Čuva decu, ruter i prefiks ako se neki prefiks završava u tom čvoru
`IPTrie.dodaj_prefiks`	Pretvara mrežu u bitove i ubacuje samo deo koji pripada maski
`IPTrie.nadji_najduzi_prefiks`	Pronalazi najkonkretniji prefiks za unetu IP adresu
`dijkstra`	Računa najkraću putanju kroz graf rutera
`simuliraj_rutiranje`	Spaja rezultat trie pretrage i Dijkstra algoritma

Za rad sa IP adresama koristi se standardni Python modul `ipaddress`. To uklanja potrebu da se ručno proverava ispravnost adrese i da se ručno računa mrežna adresa. Na primer, mreža 192.168.2.0/24 se pretvara u 32-bitni zapis, ali se u trie ubacuje samo prvih 24 bita.

5.1 Primer ruting tabele

IP prefiks	Odredišni ruter
192.168.1.0/24	R4
192.168.2.0/24	R5
10.0.0.0/8	R3
172.16.0.0/16	R2

Ova tabela je mala, ali pokazuje osnovnu ideju. U realnim mrežama broj prefiksa može biti veoma veliki, pa linearno pretraživanje tabele ne bi bilo dobro rešenje. Trie omogućava da pretraga zavisi od dužinu adrese, a ne direktno od broja unetih prefiksa.

6 Analiza vremenske složenosti

Vremenska složenost zavisi od dve veličine: dužine IP adrese, odnosno broja bitova koji se obilaze u trie strukturi, i veličine grafa rutera. Za IPv4 adrese dužina je uvek 32 bita, pa se u praktičnom smislu pretraga prefiksa može smatrati konstantnom. U opštijoj analizi, ako je L broj bitova, pretraga traje $O(L)$.

Operacija	Složenost	Objašnjenje
Ubacivanje prefiksa u trie	$O(p)$	p je dužina prefiksa, najviše 32 za IPv4
Pretraga najdužeg prefiksa	$O(L)$	L je broj bitova adrese, 32 za IPv4
Dijkstra sa heap strukturom	$O((V+E)\log V)$	V je broj rutera, E broj veza
Ukupno rutiranje	$O(L+(V+E)\log V)$	Trie bira odredišni ruter, Dijkstra bira putanju
Prostor za trie	$O(N \cdot L)$	N je broj prefiksa, L maksimalna dužina prefiksa
Prostor za graf	$O(V+E)$	Lista susedstva čuva rutere i veze

Najvažniji zaključak je da se problem razdvaja na dva manja problema. Bez trie strukture, najjednostavnije rešenje bi za svaku IP adresu prolazilo kroz celu ruting tabelu i proveravalo koji je prefiks najduži. To bi imalo složenost $O(N \cdot L)$, gde je N broj prefiksa. Trie smanjuje ovaj deo na $O(L)$, jer se prati samo jedna putanja kroz stablo.

Dijkstra algoritam dominira ukupnom složenošću kada graf ima mnogo rutera i veza. U studentskoj implementaciji putanja se čuva direktno u prioritetnom redu, što je čitljivo i jednostavno. Za velike mreže bi se umesto toga obično čuvao niz prethodnika, pa bi se putanja rekonstruisala tek na kraju.

7 Primer izvršavanja programa

Neka je početni ruter R1, a odredišna IP adresa 192.168.2.20. Trie pretraga prolazi kroz bitove ove adrese i pronalazi da se ona poklapa sa prefiksom 192.168.2.0/24. Taj prefiks je u ruting tabeli vezan za ruter R5.

Nakon toga Dijkstra algoritam traži najkraći put od R1 do R5. Za date cene veza dobija se putanja R1 -> R2 -> R3 -> R4 -> R5, sa ukupnom cenom 7. Program zatim ispisuje odredišnu adresu, pronađeni prefiks, odredišni ruter, najkraću putanju i ukupnu cenu.

```
Pocetni ruter: R1
Odredisna IP adresa: 192.168.2.20
Pronadjen prefiks: 192.168.2.0/24
Mreza je povezana sa ruterom: R5
Najkraca putanja: R1 -> R2 -> R3 -> R4 -> R5
Ukupna cena puta: 7
```

8 Zaključak

Kombinacija trie strukture i Dijkstra algoritma dobro opisuje osnovnu logiku rutiranja. Trie efikasno rešava problem izbora rute na osnovu IP prefiksa, dok Dijkstra efikasno rešava problem pronalaženja najkraće putanje kroz mrežu. Iako je program pojednostavljen, on prikazuje dve važne ideje: izbor odgovarajuće strukture podataka za brzo pretraživanje i izbor odgovarajućeg algoritma grafa za optimizaciju putanje.

Reference

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press.

[2] J. F. Kurose, K. W. Ross, *Computer Networking: A Top-Down Approach*, Pearson.

[3] Python dokumentacija: moduli `ipaddress` i `heapq`.